GESTIÓN DE LA CALIDAD DEL SOFTWARE · UFG

Monedero doc

Monedero electrónico prepago de circuito cerrado — prototipo de demostración

Este documento resume qué es el proyecto, cómo está construido, cómo funciona el backend y cómo se ven las vistas de cada rol. Está dividido en tres partes para tres expositores (unos 3 minutos cada una) y termina con la demostración y los próximos pasos.

Parte	Tema
1	El proyecto, el stack y cómo se trabajó
2	El backend: API, reglas del dinero y pruebas
3	Las vistas de los roles, la demostración y los próximos pasos

Repositorio: github.com/dn223/Monedero

Parte 1 · El proyecto, el stack y cómo se trabajó

Expositor 1 · ≈ 3 minutos

1.1 Qué es Monedero

Una tarjeta prepago como la de un supermercado: se **recarga en caja** y se **gasta solo dentro del comercio** (circuito cerrado). Cada tarjeta tiene un número único de 16 dígitos y un saldo. Aunque es académico, se diseñó **como si manejara dinero real**: no se puede perder ni duplicar un centavo.

1.2 Usuarios

- **Titular:** ve su saldo, su historial y su tarjeta virtual con QR; reporta la pérdida y cambia su PIN.
- **Caja:** lee la tarjeta, cobra (con PIN si supera $25) y recarga en múltiplos de $5.
- **Administración:** emite y repone tarjetas, gestiona las cajas y revisa la conciliación.

1.3 Stack

Parte	Tecnología	Para qué
App	Flutter (Dart), Material 3, Riverpod, go_router	Una sola app para Android y web
API	Dart con shelf	Aplica las reglas y la seguridad
Base de datos	PostgreSQL 17	Única fuente de verdad del saldo
Código compartido	Paquete *core* en Dart	App y API validan igual
Entorno local	Docker Compose y Mailpit	Todo corre en la laptop con un comando, a costo $0
Producción (diseñada)	VPS Contabo, Caddy, pgBackRest a Backblaze B2	HTTPS y respaldos continuos
Tarjetas	QR simulado hoy · NFC MIFARE Classic después	Leer número, UID y firma

1.4 Arquitectura



La app nunca decide el saldo: envía cada operación a la API, que revisa las reglas, y PostgreSQL guarda el resultado y se protege a sí misma.

1.5 Cómo se trabajó

- **Monorepo** con la app, la API, el paquete compartido, la base de datos y la documentación en un solo repositorio de **GitHub**.
- Trabajo por **fases** con entregables verificables; cada fase termina con pruebas en verde, documentación y commit.

- **Ramas y Conventional Commits**, **integración continua** (GitHub Actions corre análisis y pruebas en cada cambio).
- **ADR** (registro de decisiones técnicas): por ejemplo, PostgreSQL en lugar de Firestore y montos en centavos.
- **AGENTS.md**: instrucciones únicas para que cada integrante trabaje con su propia IA sin romper las reglas; y una **bitácora** de sesiones.

Parte 2 · El backend: API, reglas del dinero y pruebas

Expositor 2 · ≈ 3 minutos

2.1 Reglas del dinero

- Montos en **centavos enteros** (BIGINT / int), nunca decimales: evita errores de redondeo.
- Recargas solo en **múltiplos de $5**; cobros mayores que cero y **sin sobregiro**; **PIN** si el cobro supera $25.
- Solo una tarjeta **ACTIVE** puede recibir recargas o cobros (estados: PENDING, ACTIVE, BLOCKED, REPLACED).
- Cada tarjeta lleva un **UID** y una **firma HMAC** que solo el servidor puede generar: nadie puede fabricar una tarjeta válida.

2.2 Cómo funciona un cobro

La caja envía el cobro con una **clave de idempotencia**. La API valida el rol, la caja autorizada, la firma de la tarjeta y el monto. Dentro de una sola **transacción**, bloquea la tarjeta (*SELECT … FOR UPDATE*), revisa el saldo, lo actualiza y registra el movimiento. Si la red falla y la caja reintenta con la misma clave, recibe el mismo comprobante sin cobrar dos veces.

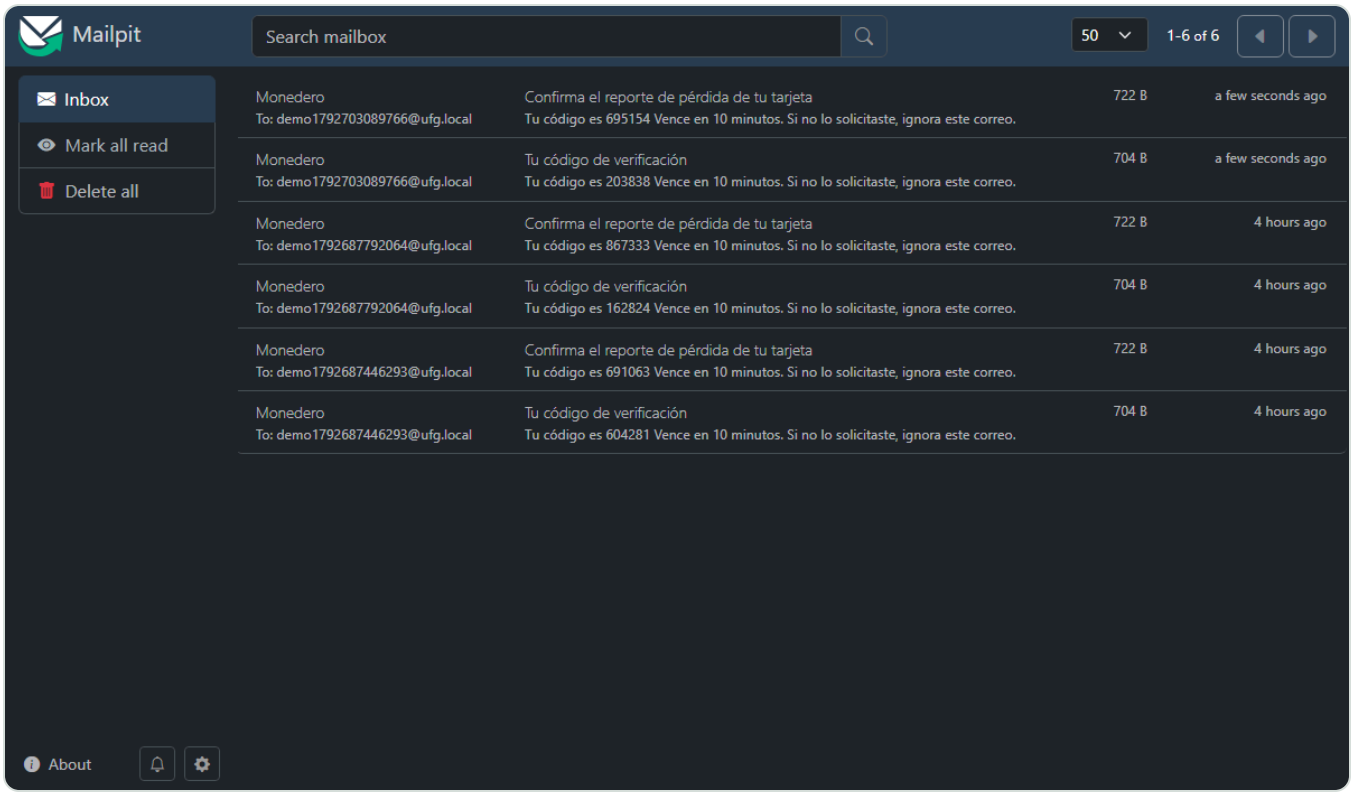
2.3 Tres capas de protección

- **App:** valida para ayudar al usuario, pero no decide.
- **API:** es la autoridad (roles, firma, saldo, PIN, idempotencia).
- **Base de datos:** última barrera: *CHECK* que impide saldos negativos, movimientos de **solo inserción** (sin permisos de modificar y un trigger que lo bloquea) y una vista de **conciliación** (saldo = suma de movimientos).

2.4 Lo que expone la API

Rol	Operaciones
Público	Registro del titular, verificación del correo con código, inicio de sesión
Titular	Mis tarjetas, historial, reportar pérdida con código, cambiar PIN
Caja	Leer tarjeta, recargar y cobrar (con credencial de la caja)
Administración	Emitir y entregar tarjetas, reponer, buscar, conciliación y cajas

Los códigos de verificación se envían por correo a **Mailpit**, un buzón local que no envía nada a Internet:



2.5 Pruebas que demuestran el diseño

33 pruebas automáticas en verde. Las 5 obligatorias corren contra PostgreSQL real:

- **1,000 cobros simultáneos** a la misma tarjeta: 400 aplicados, 600 rechazados por saldo, nunca saldo negativo y conciliación en cero.
- La misma operación enviada **5 veces** se aplica **una sola vez**.
- Una **reposición con falla forzada** a la mitad se revierte completa.
- Intentar **modificar o borrar** un movimiento: la base lo rechaza.
- **Firma inválida** o UID cambiado: la operación se rechaza.

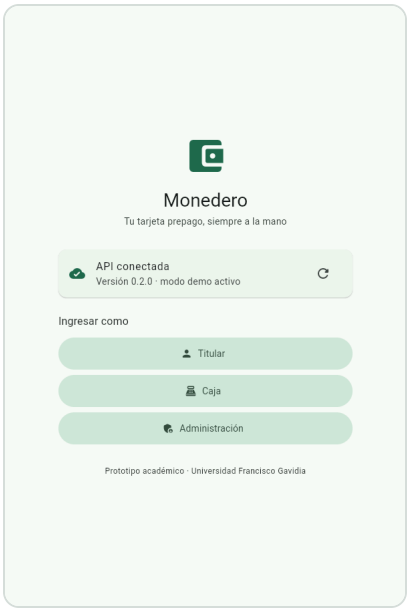
```
== Pruebas: packages/core    +15: All tests passed!
== Pruebas: api              +16: All tests passed!
== Pruebas: apps/app          +3: All tests passed!
  [ok] Sin double/num ni tipos decimales para dinero.
  [ok] Todo en verde.
```

Parte 3 · Vistas de los roles, demostración y próximos pasos

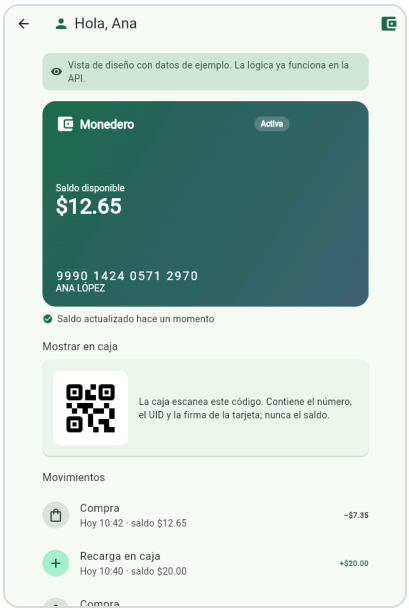
Expositor 3 · ≈ 3 minutos

3.1 Vistas de la app

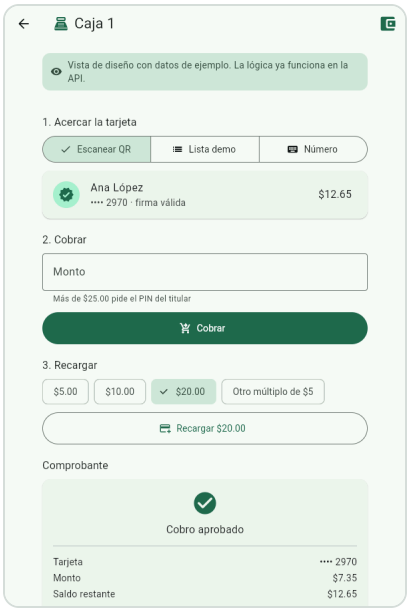
Una sola app con tema claro y oscuro. La pantalla inicial confirma que la API está conectada y da acceso a los tres roles. Hoy son **vistas de diseño con datos de ejemplo**; la lógica real ya funciona en la API.



Inicio

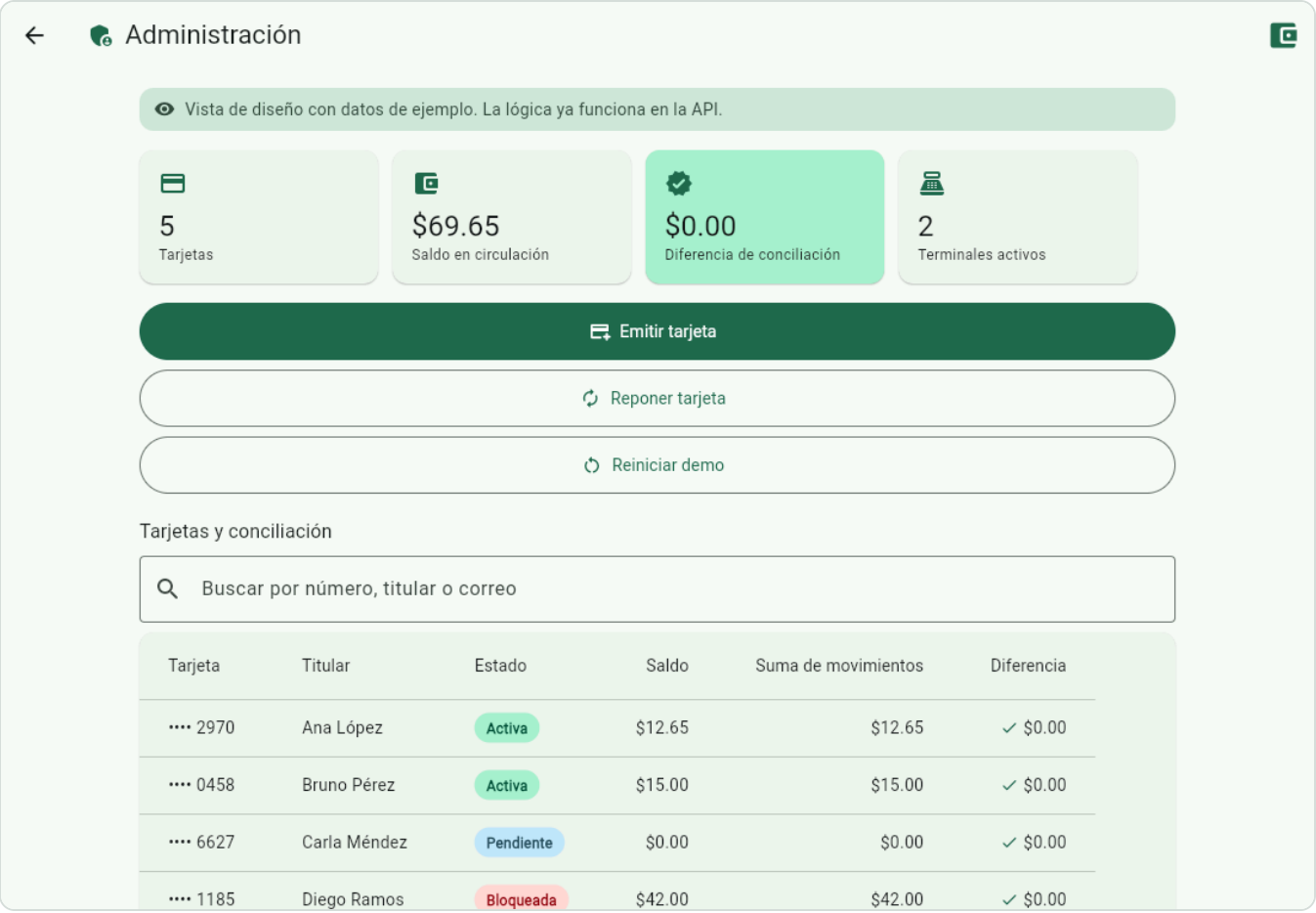


Titular



Caja

- **Titular:** tarjeta virtual con saldo, QR para mostrar en caja (número, UID y firma; nunca el saldo), movimientos, cambiar PIN y reportar pérdida.
- **Caja:** tres formas de leer la tarjeta (QR, lista demo, número), cobrar, recargar con $5, $10, $20 u otro múltiplo, comprobante y mensajes de error claros.
- **Administración:** resumen con la diferencia de conciliación, tabla de tarjetas con estados, emitir, reponer, cajas y reiniciar demo.



Administración

3.2 Demostración

Se ejecuta **dart run tool/demo.dart --pausa**: hace las mismas llamadas que hará la app, contra la API y la base de datos reales. Salida real de una corrida:

```
── Paso 1. Registrar un titular
   Código recibido en Mailpit: 203838
── Paso 2. Emitir y entregar una tarjeta virtual
   Número: 9990829260569814 · UID: 7C260EAB88827A
   Firma HMAC: 7802AC1AD4B6BA08021F80C12FE0FEB8 · PENDING → ACTIVE
── Paso 3. Recargar $20.00          Comprobante: $20.00
── Paso 4. Cobrar $7.35             Saldo después del cobro: $12.65
── Paso 5. Sobregiro de $15.00      Respuesta: 422 INSUFFICIENT_FUNDS
   Extra: misma recarga 5 veces     Solo se sumó $5.00 una vez: $17.65
   Extra: tarjeta falsificada       Respuesta: 422 INVALID_CARD_SIGNATURE
── Paso 6. Reportar la pérdida      Estado: BLOCKED (cobro posterior: 409 CARD_NOT_ACTIVE)
── Paso 7. Reponer la tarjeta       Saldo trasladado: $17.65 · nueva ACTIVE · anterior REPLACED
── Paso 8. Conciliación             Tarjetas con diferencia: 0 · Diferencia total: $0.00
   ✓ Conciliación en cero: saldo = suma de movimientos en todas las tarjetas.
```

3.3 Próximos pasos

Fase	Contenido	Estado
1–2	Estructura, base de datos, API y pruebas obligatorias	Terminada

3	Vistas de diseño de los tres roles	Terminada
4	Conectar titular y caja a la API: inicio de sesión, saldo real, QR real, lector de tarjetas, cobro y recarga	Siguiente
5	Conectar administración: emitir, buscar, conciliación, cajas, reposición y reiniciar demo	Pendiente
6	App en Android, pulido, guion final y despliegue preparado	Pendiente

Después del prototipo: **tarjetas físicas NFC** (MIFARE Classic 1K), **servidor en Contabo** con HTTPS y **respaldos continuos**, y la elección del **nombre definitivo** de la app.